

Q4_functional_classification

September 7, 2026

1 ISyE 6525 HW1 - Question 4

Goal: classify acoustic-emission signals as sharp (class 0) or worn (class 1).

- Part 1: cubic B-spline coefficients with 10 knots → RBF SVM.
- Part 2: FPCA scores → RBF SVM using 2, 5, 8, and 10 harmonics.
- Part 3: compare the feature representations.

```
[1]: from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.decomposition import PCA
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score, \
    ↪confusion_matrix
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from patsy import dmatrix

TRAIN_PATH = Path('Question4_train.csv')
TEST_PATH = Path('Question4_test.csv')
N_GRID = 128
RANDOM_STATE = 42
```

1.1 Load and inspect the signals

The first 128 columns are signal values on an equally spaced grid $t \in [0, 1]$; the last column is the class label.

```
[2]: for path in (TRAIN_PATH, TEST_PATH):
    if not path.exists():
        raise FileNotFoundError(f'Put {path.name} beside this notebook.')

train = pd.read_csv(TRAIN_PATH, header=None).to_numpy(dtype=float)
test = pd.read_csv(TEST_PATH, header=None).to_numpy(dtype=float)
```

```

assert train.shape == (120, 129), f'Expected train shape (120, 129), got {train.
    ↪shape}'
assert test.shape == (60, 129), f'Expected test shape (60, 129), got {test.
    ↪shape}'

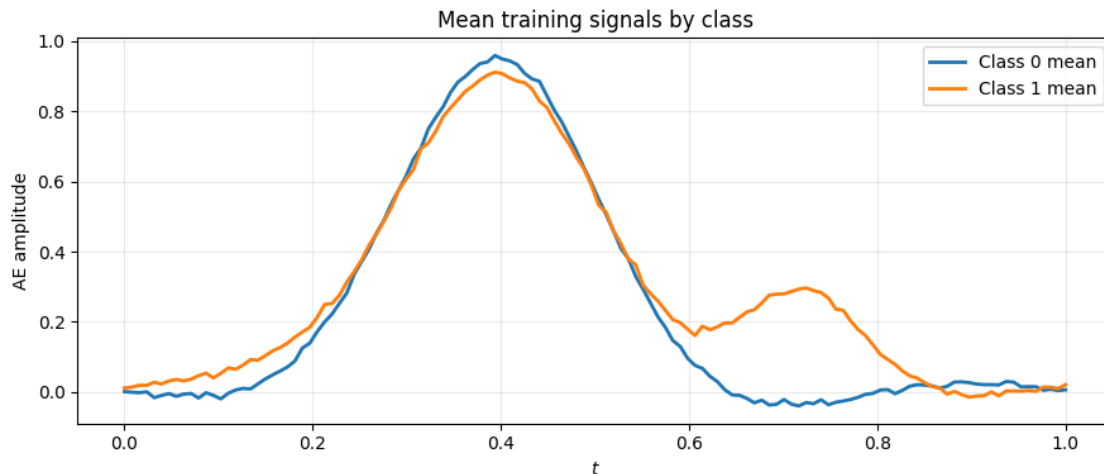
X_train, y_train = train[:, :N_GRID], train[:, -1].astype(int)
X_test, y_test = test[:, :N_GRID], test[:, -1].astype(int)
t = np.linspace(0.0, 1.0, N_GRID)

print('Train:', X_train.shape, ' Test:', X_test.shape)
print('Training class counts:', np.bincount(y_train))

plt.figure(figsize=(9, 4))
for label, color in [(0, 'tab:blue'), (1, 'tab:orange')]:
    plt.plot(t, X_train[y_train == label].mean(axis=0), color=color,
             linewidth=2, label=f'Class {label} mean')
plt.xlabel('$t$')
plt.ylabel('AE amplitude')
plt.title('Mean training signals by class')
plt.grid(alpha=0.25)
plt.legend()
plt.tight_layout()
plt.show()

```

Train: (120, 128) Test: (60, 128)
 Training class counts: [60 60]



1.2 Part 1: cubic B-spline coefficients + RBF SVM

Construct one common B-spline basis matrix Φ on the 128-point grid, then estimate each signal's coefficients by least squares:

$$x_i(t) \approx \sum_j c_{ij} B_j(t).$$

```
[7]: N_TOTAL_KNOTS = 10
interior_knots = np.linspace(0.0, 1.0, N_TOTAL_KNOTS)[1:-1]
basis_formula = (
    f'bs(t, knots={interior_knots.tolist()}, degree=3, '
    'include_intercept=True, lower_bound=0, upper_bound=1) - 1'
)
Phi = np.asarray(dmatrix(basis_formula, {'t': t}))
# Phi shape: grid points x basis functions

def spline_coefficients(signals, basis_matrix):
    # Solve Phi @ coefficient_vector = signal for every row/signal.
    coefficients, *_ = np.linalg.lstsq(basis_matrix, signals.T, rcond=None)
    return coefficients.T

C_train = spline_coefficients(X_train, Phi)
C_test = spline_coefficients(X_test, Phi)
print('B-spline feature shape:', C_train.shape)

bspline_svm = make_pipeline(StandardScaler(), SVC(kernel='rbf'))
bspline_svm.fit(C_train, y_train)
bspline_pred = bspline_svm.predict(C_test)

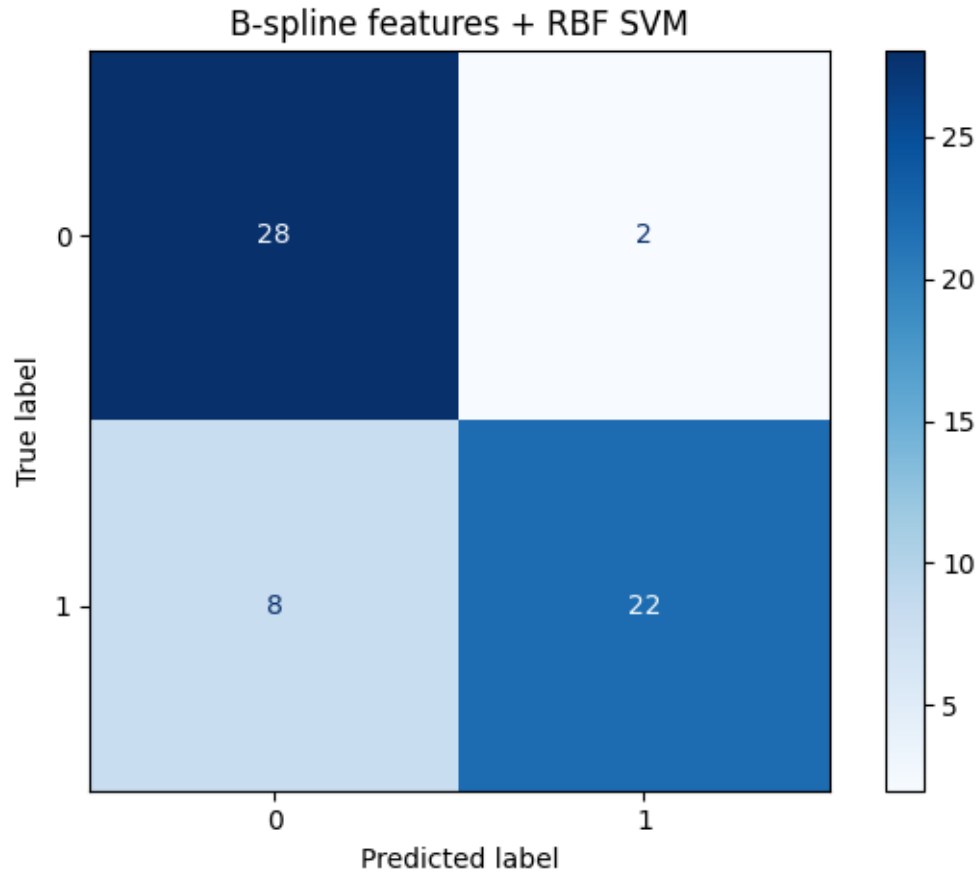
bspline_accuracy = accuracy_score(y_test, bspline_pred)
bspline_cm = confusion_matrix(y_test, bspline_pred)
print(f'Test accuracy: {bspline_accuracy:.3f}')
print('Confusion matrix:\n', bspline_cm)
ConfusionMatrixDisplay(bspline_cm).plot(cmap='Blues')
plt.title('B-spline features + RBF SVM')
plt.tight_layout()
plt.show()
```

B-spline feature shape: (120, 12)

Test accuracy: 0.833

Confusion matrix:

```
[[28  2]
 [ 8 22]]
```



1.3 Part 2: FPCA features

On this shared dense grid, PCA of the centered signal vectors gives the discretized FPCA solution. Fit PCA on the **training signals only**, and use the fitted transformation for the test signals.

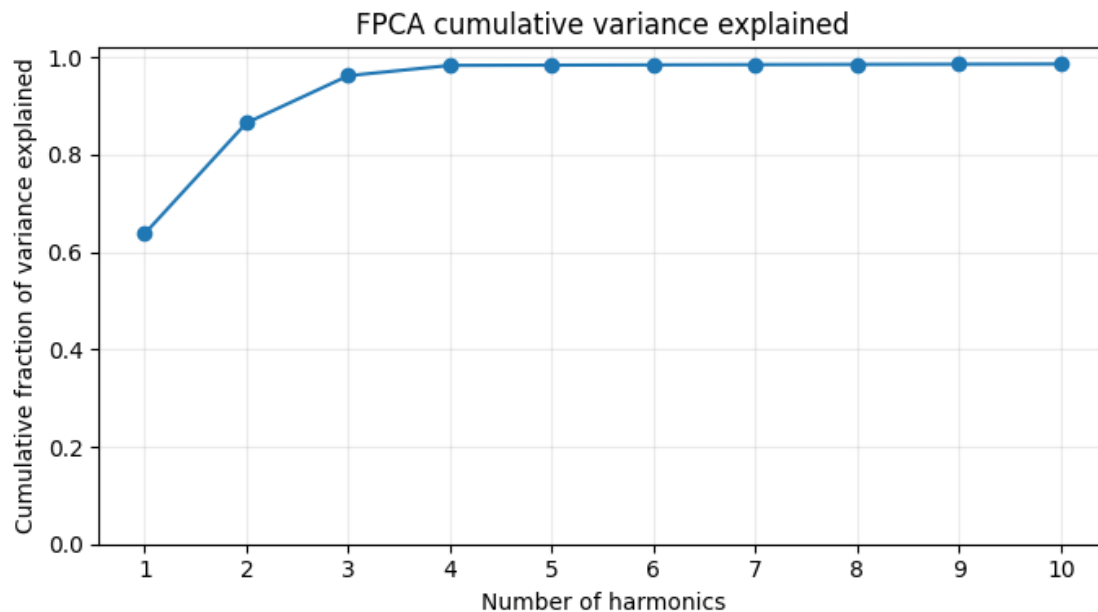
```
[4]: fpca = PCA(n_components=10, random_state=RANDOM_STATE)
scores_train = fpca.fit_transform(X_train)
scores_test = fpca.transform(X_test)

fve = fpca.explained_variance_ratio_
cumulative_fve = np.cumsum(fve)
fve_table = pd.DataFrame({
    'Harmonic': np.arange(1, 11),
    'Fraction variance explained': fve,
    'Cumulative fraction': cumulative_fve,
})
display(fve_table)

plt.figure(figsize=(7, 4))
```

```
plt.plot(np.arange(1, 11), cumulative_fve, marker='o')
plt.xticks(np.arange(1, 11))
plt.ylim(0, 1.02)
plt.xlabel('Number of harmonics')
plt.ylabel('Cumulative fraction of variance explained')
plt.title('FPCA cumulative variance explained')
plt.grid(alpha=0.25)
plt.tight_layout()
plt.show()
```

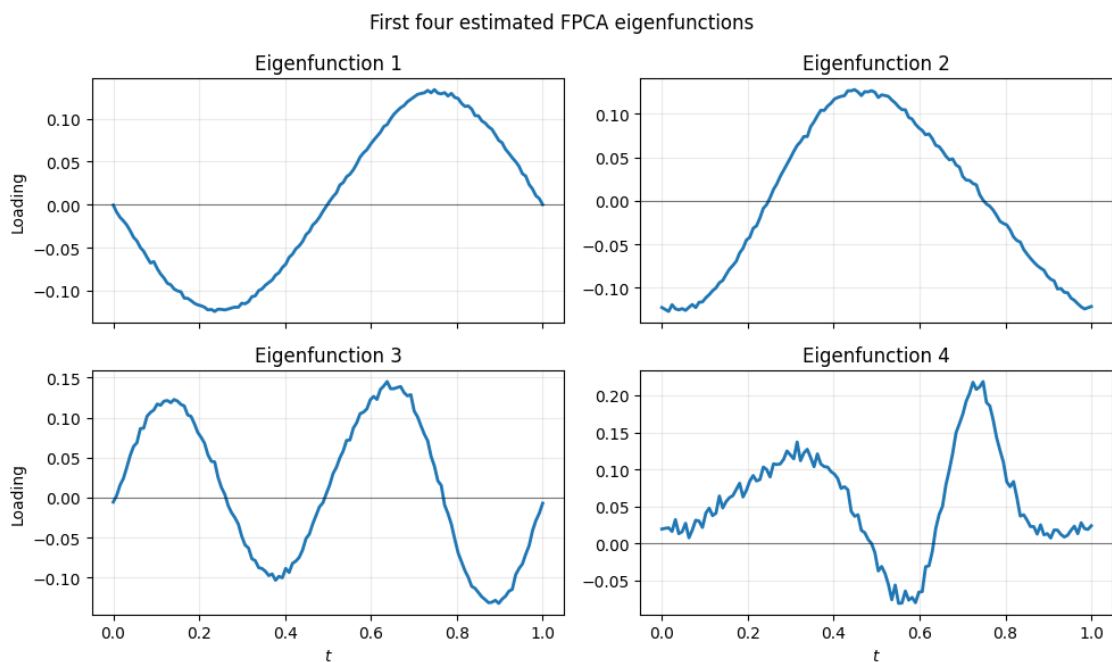
	Harmonic	Fraction variance explained	Cumulative fraction
0	1	0.637779	0.637779
1	2	0.227651	0.865430
2	3	0.097105	0.962535
3	4	0.021164	0.983699
4	5	0.000562	0.984261
5	6	0.000514	0.984774
6	7	0.000469	0.985243
7	8	0.000460	0.985703
8	9	0.000440	0.986143
9	10	0.000432	0.986575



1.3.1 First four estimated eigenfunctions

The sign of an eigenfunction is arbitrary: multiplying an eigenfunction and all of its scores by -1 represents the same component.

```
[5]: fig, axes = plt.subplots(2, 2, figsize=(10, 6), sharex=True)
for k, ax in enumerate(axes.flat[:4]):
    ax.plot(t, fpca.components_[k], linewidth=2)
    ax.axhline(0, color='black', linewidth=0.8, alpha=0.5)
    ax.set_title(f'Eigenfunction {k + 1}')
    ax.grid(alpha=0.25)
for ax in axes[-1, :]:
    ax.set_xlabel('$t$')
for ax in axes[:, 0]:
    ax.set_ylabel('Loading')
fig.suptitle('First four estimated FPCA eigenfunctions')
fig.tight_layout()
plt.show()
```



1.3.2 RBF SVMs using 2, 5, 8, and 10 scores

TODO: report both accuracy and confusion matrix for every value of m . If tuning SVM hyperparameters, tune separately using training-only CV.

```
[6]: n_harmonics_options = [2, 5, 8, 10]
fpca_results = []
fpca_confusion_matrices = {}

for m in n_harmonics_options:
    model = make_pipeline(StandardScaler(), SVC(kernel='rbf'))
    model.fit(scores_train[:, :m], y_train)
```

```

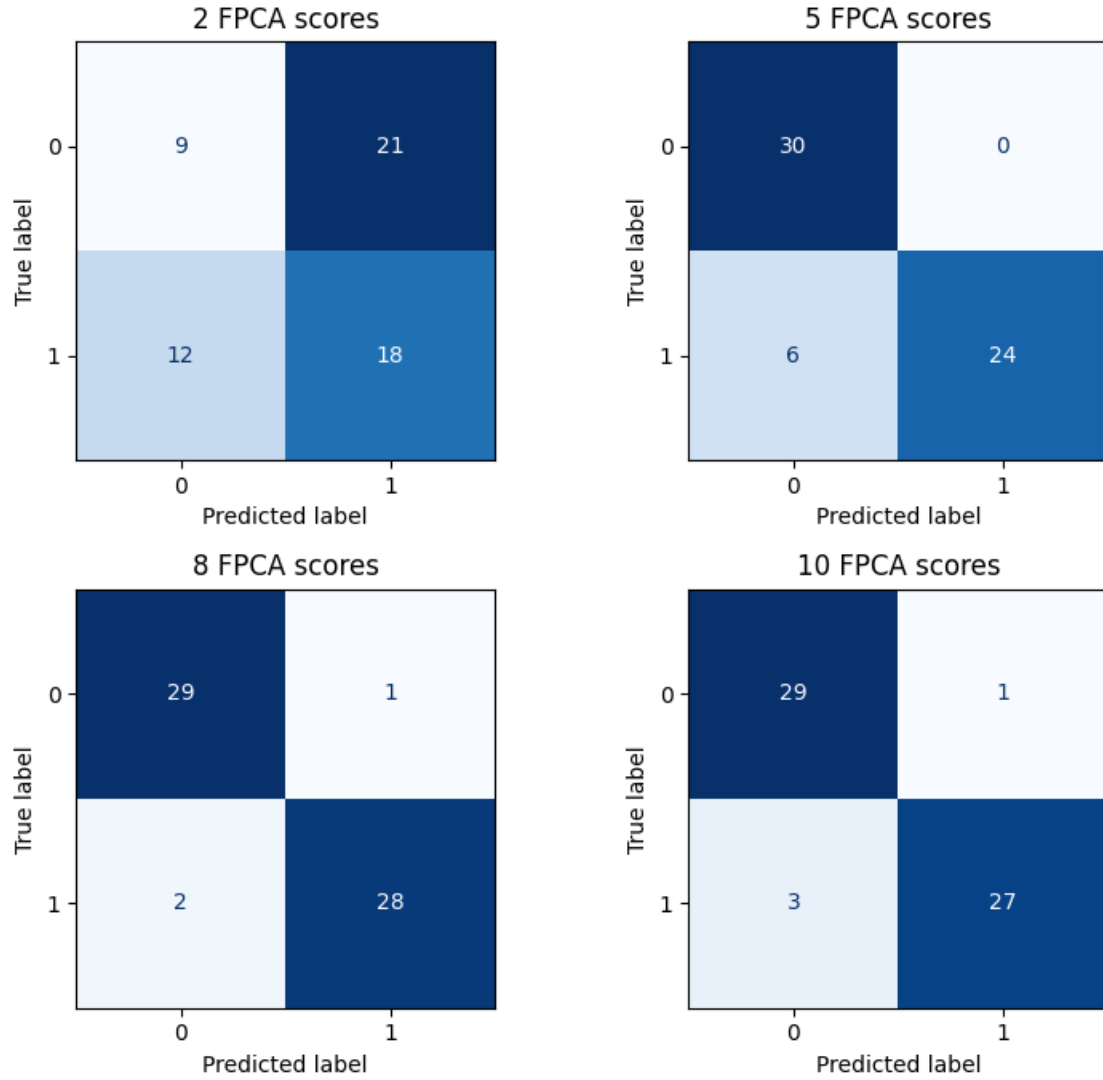
pred = model.predict(scores_test[:, :m])
acc = accuracy_score(y_test, pred)
cm = confusion_matrix(y_test, pred)
fpca_results.append({'Harmonics': m, 'Test accuracy': acc})
fpca_confusion_matrices[m] = cm

display(pd.DataFrame(fpca_results))

fig, axes = plt.subplots(2, 2, figsize=(8, 7))
for m, ax in zip(n_harmonics_options, axes.flat):
    ConfusionMatrixDisplay(fpca_confusion_matrices[m]).plot(
        ax=ax, cmap='Blues', colorbar=False
    )
    ax.set_title(f'{m} FPCA scores')
fig.tight_layout()
plt.show()

```

	Harmonics	Test accuracy
0	2	0.450000
1	5	0.900000
2	8	0.950000
3	10	0.933333



1.4 Part 3: comparison

TODO: compare the B-spline result with the four FPCA results in two or three sentences. Mention that FPCA orders components by total variance, not class-separating power. A small late-cycle wear signal may appear in a later harmonic, so retaining too few harmonics can discard useful classification information.